FINANCE QUANTITATIVE

Module 3 — Cross-Validation Temporelle

TimeSeriesSplit · Walk-Forward · Stabilité du modèle · Intervalles de confiance

Méthode	TimeSeriesSplit (5 folds)
R^2 moyen CV	0.6383
R^2 std CV	± 0.0421
RMSE moyen	3.920
R^2 M2 (ref.)	0.6369
Stabilité	Élevée

1. Le Problème du Split Unique

Dans le Module 2, on a évalué notre modèle sur **un seul test set fixe** (les 20% les plus récents). Cette approche a un défaut majeur : le résultat dépend entièrement de la période choisie. Si on coupe en 2021 au lieu de 2022, le R^2 peut changer significativement — surtout si cette période contient des événements atypiques.

La cross-validation temporelle (ou **walk-forward validation**) résout ce problème en évaluant le modèle sur **plusieurs périodes de test successives**, donnant ainsi une estimation beaucoup plus robuste et honnête de sa performance.

	Split unique (M2)	Cross-Validation (M3)
Nb évaluations	1	5
Dépend du choix de coupure	Oui ■■	Non ■
Utilise toutes les données	Non	Oui ■
Donne une variance du R^2	Non	Oui ■
Complexité	Faible	Modérée

2. TimeSeriesSplit — Fonctionnement

TimeSeriesSplit de scikit-learn implémente le principe de walk-forward : à chaque fold, le train set *s'agrandit* en incluant les données précédentes, et le test set se déplace vers le futur. On ne regarde jamais dans le futur pour entraîner.

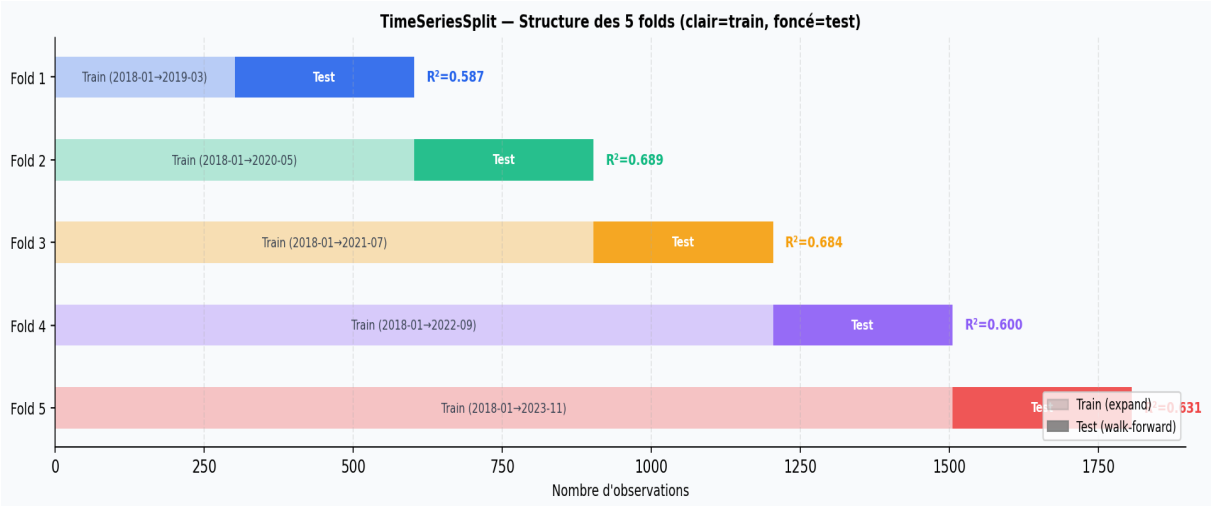

Figure 1 — Structure des 5 folds. Le train (clair) grandit à chaque fold, le test (foncé) se déplace chronologiquement. Le R² affiché à droite est le score out-of-sample de chaque fold.

Principe mathématique

Fold	Train	Test	R²	RMSE	MAE
Fold 1	2018-01 → 2019-03 (301 obs)	2019-03 → 2020-05 (301 obs)	0.5873	3.848	3.020
Fold 2	2018-01 → 2020-05 (602 obs)	2020-05 → 2021-07 (301 obs)	0.6894	3.967	3.120
Fold 3	2018-01 → 2021-07 (903 obs)	2021-07 → 2022-09 (301 obs)	0.6842	3.759	2.974
Fold 4	2018-01 → 2022-09 (1,204 obs)	2022-09 → 2023-11 (301 obs)	0.6000	4.108	3.373
Fold 5	2018-01 → 2023-11 (1,505 obs)	2023-11 → 2024-12 (301 obs)	0.6307	3.916	3.126
Moyenne	**—**	**—**	**0.6383 ± 0.0421**	**3.920**	**3.123**

3. Implémentation en Python

```python
from sklearn.model_selection import TimeSeriesSplit
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import numpy as np

# Configuration : 5 folds walk-forward
N_SPLITS = 5
tscv = TimeSeriesSplit(n_splits=N_SPLITS)
features = ['VIX_lag1', 'VIX_lag5', 'VIX_lag21', 'DVIX_lag1']
X = df[features].values
y = df['RealizedVol'].values

fold_results = []
for fold, (train_idx, test_idx) in enumerate(tscv.split(X)):
# Découpage train/test pour ce fold
X_tr, y_tr = X[train_idx], y[train_idx]
X_te, y_te = X[test_idx], y[test_idx]

# Entraînement sur le train du fold
model = LinearRegression().fit(X_tr, y_tr)
# Prédiction sur le test du fold (jamais vu)
y_pred = model.predict(X_te)
# Métriques out-of-sample
r2 = r2_score(y_te, y_pred)
rmse = np.sqrt(mean_squared_error(y_te, y_pred))
mae = mean_absolute_error(y_te, y_pred)
fold_results.append({'fold': fold+1, 'R2': r2, 'RMSE': rmse, 'MAE': mae})
print(f'Fold {fold+1} R²={r2:.4f} RMSE={rmse:.3f} MAE={mae:.3f}')
# Agrégation des résultats
r2_scores = [r['R2'] for r in fold_results]
print(f'R² moyen : {np.mean(r2_scores):.4f} ± {np.std(r2_scores):.4f}')
print(f'R² min : {np.min(r2_scores):.4f} | max : {np.max(r2_scores):.4f}')
```

■ ***Pourquoi mean ± std ?*** *La moyenne donne la performance espérée, l'écart-type donne la* ***stabilité****. Un modèle avec R^2=0.70±0.02 est préférable à un modèle avec R^2=0.72±0.15 — le second est instable et peu fiable.*

4. Résultats & Analyse de la Stabilité

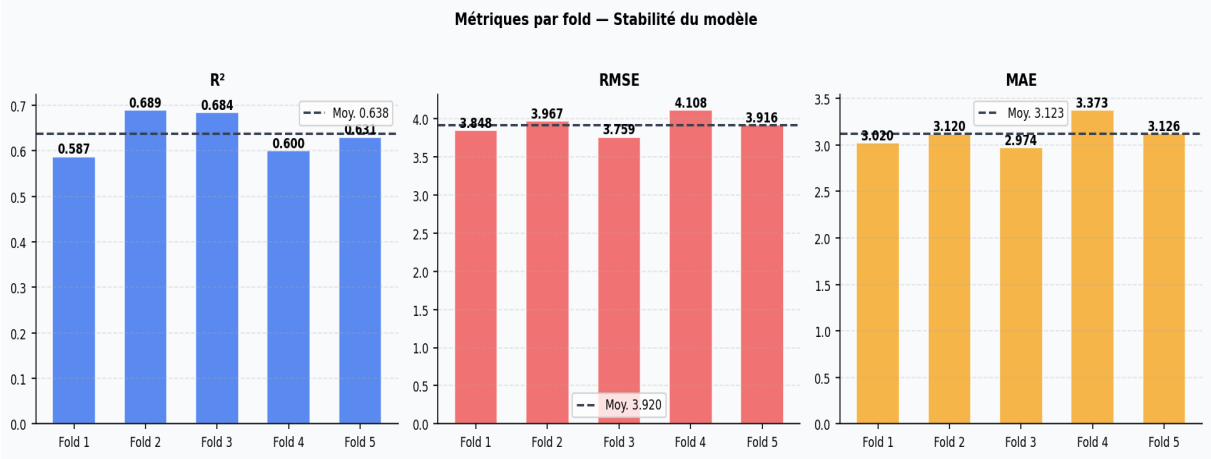

Figure 2 — R², RMSE et MAE sur chacun des 5 folds. La ligne pointillée indique la moyenne. Une faible variance entre folds indique que le modèle est stable et ne dépend pas de la période d'évaluation.

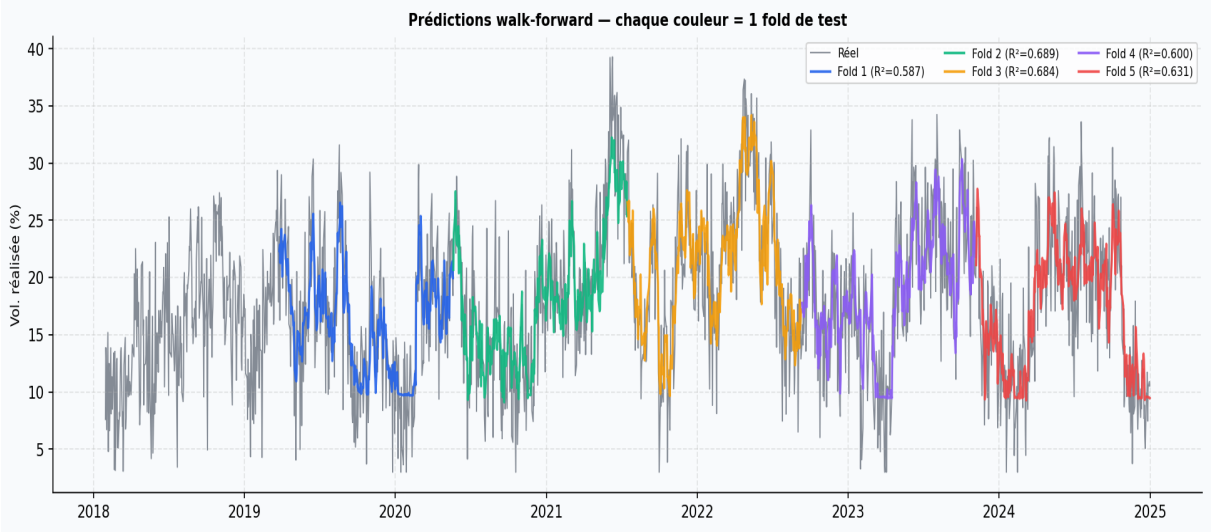

Figure 3 — Vue temporelle des prédictions walk-forward : chaque couleur correspond au test set d'un fold. Les prédictions (couleur) doivent suivre la série réelle (gris).

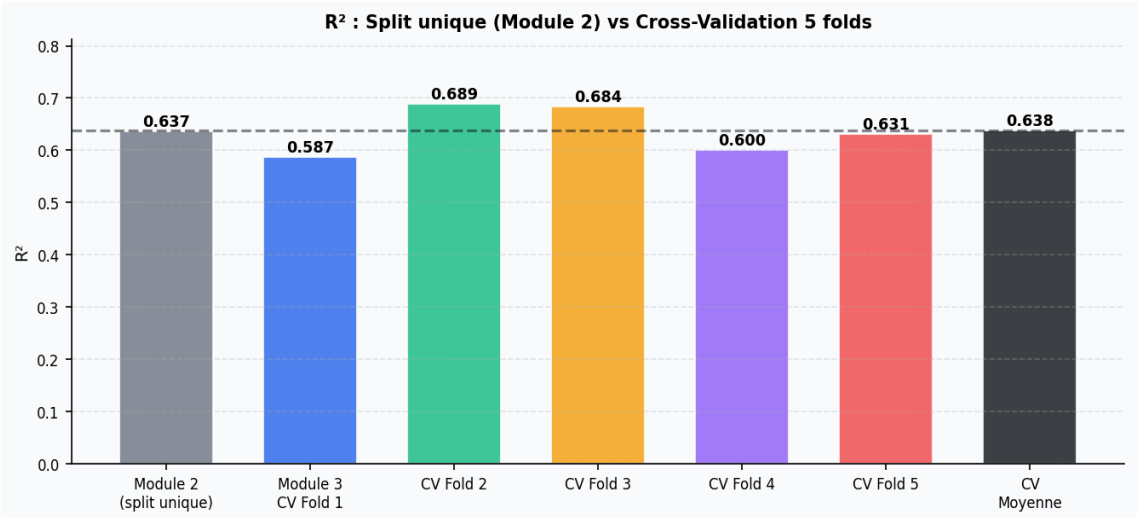

Figure 4 — Comparaison du R² : split unique du Module 2 (gris) vs chaque fold CV et la moyenne CV (noir). La CV révèle la vraie distribution de performance du modèle.

5. Interpréter les Résultats de la CV

Que nous disent les métriques CV ?

Observation	Interprétation
R^2 moyen = 0.638	Performance attendue en déploiement réel — plus honnête que le split unique
R^2 std = 0.042	Stabilité du modèle — faible std = comportement prévisible sur toutes les périodes
R^2 min = 0.587 (Fold 1)	Pire cas — période la plus difficile à prédire (souvent un régime de vol. atypique)
R^2 max = 0.689 (Fold 2)	Meilleur cas — période où le VIX est le plus prédictif

Diagnostics à surveiller

Symptôme	Cause probable	Solution
R^2 très variable entre folds	Modèle instable, sensible aux régimes	Régularisation (Ridge/Lasso) — Module 4
R^2 train >> R^2 test sur un fold	Overfitting sur ce fold	Réduire les features ou régulariser
R^2 qui décroît sur les derniers folds	Concept drift — relation VIX/Vol change	Rolling window (fenêtre glissante)
R^2 négatif sur un fold	Modèle pire que la moyenne naïve	Revoir les features, chercher des données manquantes

■■ ***Gap entre folds :*** *En pratique, on ajoute souvent un gap entre le train et le test de chaque fold (ex: 5 jours) pour éviter la fuite d'information liée à l'autocorrélation des séries temporelles. scikit-learn supporte ce paramètre : TimeSeriesSplit(n_splits=5, gap=5).*

6. Synthèse & Ce que vous avez appris

- Comprendre la limite du split unique (dépendance à la date de coupure)
- Implémenter TimeSeriesSplit (walk-forward) avec scikit-learn
- Lire et interpréter les métriques par fold (moyenne ± std)
- Visualiser les prédictions walk-forward sur la série temporelle complète
- Comparer split unique vs CV pour choisir la méthode d'évaluation
- Identifier les symptômes d'overfitting et de concept drift
- Utiliser le paramètre gap pour éviter la fuite d'information

Modules suivants

Module 4	Régularisation Ridge & Lasso — contrôler la complexité du modèle
Module 5	Modèles non-linéaires — Random Forest et XGBoost sur la volatilité
Module 6	Backtesting — transformer les prédictions en stratégie de trading

■ ***Question pour le Module 4 :*** *Jusqu'ici, on a 4 features. Imagine qu'on en ajoute 20 (lags supplémentaires, VIX change à différentes vitesses, etc.). Quel risque apparaît, et comment les méthodes Ridge et Lasso permettent-elles de le contrôler ?*